

Makefile

Bài 1: Giới thiệu seri Makefile

- Mục tiêu: có thể Build project vừa và nhỏ, Không dùng IDE
 - + Sd command sinh ra file.hex, .bin từ file.c, .h, ...
- Bản chất Makefile dùng để thực thi các command
- Khi dùng Makefile để rebuild 1 dự án, nó sẽ chỉ build lại file đã thay giúp giảm tg rebuild
- Vì dòng chip ms không có IDE hỗ trợ $\Rightarrow$ dùng makefile để build ra file.b, .hex
 - (sử dụng trình biên dịch: là cốt lõi của và makefile: lên kịch bản ...)
 - ctr
- * Mục tiêu của khóa học

Bài 2: Cài đặt môi trường msys2, gcc.

- Msys2: tập hợp các tool, library cung cấp chơ để dăng sd mt trong build, install, running native Window software
 - GCC, mingw-w64, CPython, CMake, Meson, OpenSSL, FFmpeg, Rust, Ruby, ...
- GCC: là 1 compiler để build dự án.

- IDE = UI (Giao diện) + Env (Môi trường)
 $\downarrow$
(VScode, Notepad, ...)

$\Rightarrow$ Dùng Msys2 để build (Check: gcc --version)
(Tự config môi trường)

Mt: Edit Environment system: thêm path (C:\msys64\usr\bin)
ucrt64...

Bài 3: Cách build dự án = gcc, ko thông qua IDE

- Tạo folder: Test $\Rightarrow$ tạo main.c $\Rightarrow$ Code = Notepad (viết code)
- Mở Cmd

* Cach build vs gcc : gcc main.c -o main.exe

↓ ↓ ↓

file cần build output file output (hex/bin)
(chạy)

main.exe (gõ sau khi sinh ra file này)

cls (clear cmd)

* Tìm hiểu thêm ss64 cmd : có nhiều command

Bài 4: Makefile làm gì? Build project có nhiều file.c, .h ntn vs gcc

1 file: Powershell: gcc main.c -o main.exe
.\main.exe

nhiều file:

* Cách 1: Từng file.c build ra từng file.o
- Link các file.o đó lại vs nhau

Code: gcc -c main.c -o main.o → build ra file.o
gcc -c main.lib.c -o lib.o
gcc main.o lib.o -o main.exe → build ra file.exe là file để chạy của ctr.

+ Trong đó "-c": tạo ra file object
"-o": output

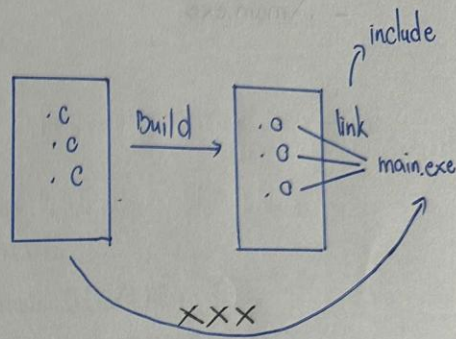
* Cách 2: Đơn giản hơn, chúng ta gộp các lệnh trên = vè dùng "-I".
gcc sẽ thực hiện tìm kiếm trong thư mục hiện tại: "." để thêm các file header ≠
(nằm ở folder root)

Code: gcc main.c lib.c -o main.exe -I. ⇒ Build ra file main.exe

+ Khi file.h nằm trong thư mục inc ⇒ Báo lỗi (do ko tìm trong file inc)

⇒ Nhận thấy cả 2 cách trên đều ko clean vs các dự án lớn

⇒ Makefile chính là để dùng cho trường hợp như vậy.



Bài 5: Viết Ctr makefile đầu tiên

- Viết file Makefile (tổng hợp các cmd của ng code)
- make main
- ./main.exe

Bài 6: Rule cơ bản trong Makefile

Target	Dependency: file phụ thuộc	
main:	main.c lib.c	
	gcc main.c lib.c -o main.exe -I.	} Rule
Tab		
	Action	

- Rule: các Rule cần thực hiện khi compiler
- Dependency: là các file cần thiết để tạo ra target
- Action: là câu lệnh compile để tạo ra Target từ Dependency
Action để thực thi vào 1 Tab (phím tab trên bàn phím) so vs Target
- Target: Là tên của nv hoặc file mà bạn muốn make tạo ra.

clean:

```
rm -f *.exe : xóa file.exe  
↓  
xóa  
make clean
```

Bài 7: Variable trong Makefile

- Khởi tạo biến trong Makefile : `NAME = VALUE`
- Call biến : `$(NAME)` hoặc `$(NAME)`

Vd: `CC = gcc`

~~`$(CC)`~~

`lib.o : lib.c`

`$(CC) -c lib.c -o lib.o -I.`

Bài 8: WILDCARDS và AUTOMATIC VARIABLE trong Makefile

`%o` : 1 pattern của 1 tệp đối tượng có phần mở rộng `.o` (file.o)

`%c` : `.c` (file.c)

⇒ Bản chất chúng được gọi là WILDCARDS: đại diện cho 1 đối tượng có phần mở rộng.

`$(@)` : là thành phần trong tệp tin bên trái dấu :

`$(<)` : đầu tiên trong danh sách của dependency

⇒ Bản chất nó được gọi là AUTOMATIC VARIABLE

`CC = gcc`

`DEPS = lib.h`

`CFLAGS = -I.`

`%o : %c $(DEPS)`

`$(CC) -c $(< -o $(@) $(CFLAGS)`

Bài 9. PHONY trong makefile

- Target: Tên file muốn tạo / Tên hàm.

+ Trong TH trong prj có tên file (clean) trùng vs tên hàm Target clean

⇒ Thì lúc này nó sẽ ko thực hiện hàm clean.

- Để tránh điều này: sửa thành . PHONY: clean

clean:

rm -f *.exe *.o

- 1 số chuẩn PHONY hay dùng

Target	Function
+ all	Thực hiện build toàn bộ
+ install	Tạo bản cài đặt của ứng dụng từ vcs compile binary
+ clean	Xoá binary file để tạo từ source
+ distclean	Xoá tất cả các file để tạo ra mà ko nằm trong source chính
+ TAGS	Tạo bảng tag để editor dùng
+ info	Tạo GNU info file từ textinfo source
+ check	Chạy bất kỳ test nào tương ứng vs ctr

- Vì makefile là viết cho ng dùng dùng nên tránh ng dùng nhầm lẫn...

Bài 10: VPATH và CFLAGS trong Makefile

- Từ trước thời điểm hiện tại, các file .c, .h đang để ở folder root nên việc thao tác vs nó rất dễ, sẽ ra sao nếu để file.c trong src
file.h inc

Code: VPATH = src (file.c)

CFLAGS = -I inc (file.h)

- Khi có này thì lúc đầu nó sẽ chủ động tìm trong folder root trước, nếu ko có các file .c, .h cần tìm thì nó sẽ tìm trong các folder mình định nghĩa ở trên
 - Khi này thì makefile sẽ tự động tìm các file .c, .h trong 2 folder ns trên và sẽ ko lỗi
- ⇒ Đây là sức mạnh của CFLAGS và VPATH

Bài 11: Foreach trong Makefile

- Trong TH có nhiều folder chứa file, h thì sao

~~Code~~ ~~INC~~

`$(foreach var, list, text)`

- Trong đó:
 - + var : Biến tạm dùng để lưu trữ 1 phần tử trong list trong 1 vòng lặp
 - + list : Danh sách các phần tử mà bạn muốn lặp qua.
 - + text: Phần tử hoặc biểu thức cần thực hiện vs 1 phần tử trong list

- Code:

INC_DIR := ./inc ./core ./a ./b (Thêm khi có thêm folder)
SRC_DIR := ./src ./core

SRC_DIR := ./src ./core

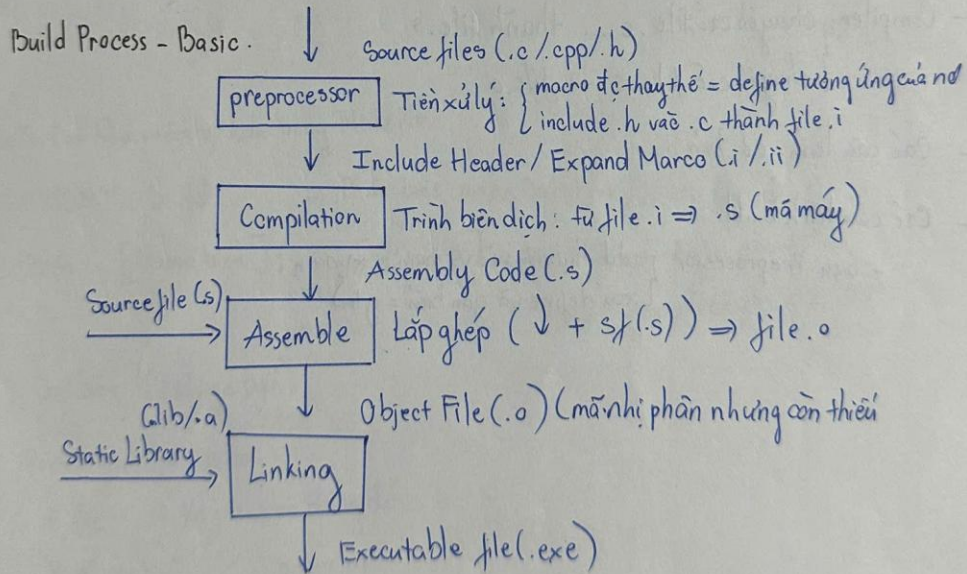
CFLAGS = \$(foreach INC_DIR, \$(INC_DIR), -I\$(INC_DIR))

```
VPATH = $(foreach SRC_DIR, $(SRC_DIRS), $(SRC_DIR))
```

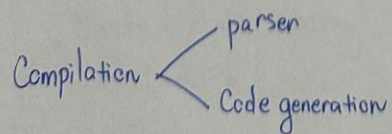
Makefile 2

Bài 1: Giới thiệu khoá học

Bài 2: Build Process 1 Ctr C



Build Process - Embedded



phải có file để ánh xạ các địa chỉ với
vì build trên máy tính nên là address máy tính
(Xem Cross Compiler)

Locator : ánh xạ vùng nhớ của file.o trên máy tính xuống vùng nhớ của vớtk

(.ld : Linker script)

+ file elf : hỗ trợ v debug

+ .ihex / .bin : nạp xuống vớtk để thực thi code

Bài 3: GCC - Command Line

- Compiler: chuyển các file.c.... thành file.s

↳ chuyển các file.s → file.o

- Các câu lệnh cơ bản:

- Các câu lệnh build:

+ San Preprocessor

↳ add thư viện vào vs main.c
↳ xóa define và gắn biến = g true

Bài 4: Xây dựng Makefile Cơ bản vs Ctr C

Nếu dùng cmd gcc thì rất mất +g vì phải gõ lệnh

Nếu có file ms thì phải rebuild lại tất cả các file $\Rightarrow$ mệt

Makefile thì nó phải build lại cả file ms thêm th

+ Project structure (cấu trúc của prj) : gồm các file và phụ thuộc của các file

+ Các câu lệnh gcc, main...

+ lệnh make thì sẽ đọc file makefile và thực thi

Nếu thư mục nằm bên trong ... /main.c ...

-I tìm file .h -I. /inc

Bài 5: 1 số khái niệm cơ bản trong Makefile

- . PHONY: build $\Rightarrow$ Phân biệt make Build vs file tên Build

- Biến : [Name biến] := [gtr, biến] = gán 1 gtr cho 1 biến
 = gán nhiều gtr cho 1 biến
 +=

Gọi biến \$(Name biến)

- Print gtr và echo:

+ @ : sẽ ko print ra dòng đó

+ @echo "...": print ra...

+ Viết print- %:

@echo \$(\$(subst print-, , \$@))

Xem thêm link tài liệu

$\Rightarrow$ khi sử dụng make print-[Name biến]
 $\Rightarrow$ sẽ print ra gtr của biến đó

Bài 6: Ctr Makefile vs VDK (STM)

- Cài: GNUARM Toolchain - Compiler ~~cho~~ GCC cho chip ARM
- Tạo proj
- Viết Makefile
- file.map: (bản đồ bộ nhớ: xem hàm bin đặt ở địa chỉ nào)
- nostlib: thư viện chuẩn của.
- * Thêm compiler option arm gcc

Bài $\Rightarrow$ file.o, .map .elf

Bài 7: TT P2

- Nạp = Utility (CLI: doc)
- + -ME: xác toàn bộ chip
- SE: xác từng section
- rst: reset VDK (sau khi nạp code)
- p [file.hex] [Địa chỉ bắt đầu] $\begin{cases} 0x80000000 \\ \dots \text{ (bootloader)} \end{cases}$

* Nhảy led

Bài 8: Tổng quát hoá - Automatic Variables. Xem (GNU Make)

- Automatic Variable

- + $\$@$: tên file target
- + $\$^$: Tất cả tên file phụ
- + $\$<$: Tên hàng phụ đầu tiên

Bài 9: wildcard foreach

- Tạo các file INC_DIRS
SRC_DIRS

- I và wildcard $\rightarrow$ thấy cuối $\Rightarrow$ foreach
 $\downarrow$
chỉ thấy đầu tiên

- wildcard: sẽ lấy ra file

(Nếu có nhiều đường dẫn thì chỉ lấy file của đường dẫn cuối cùng thôi)

$\Rightarrow$ sử dụng foreach để lấy hết file ở các thư mục đó nên.

- Đổi PATH-OUTPUT

Bài 10: Make process

- VPATH: tự động đi tìm các file phụ $E[\boxed{.c}]$ / trong SRC_DIRS
- h / INC_DIRS
- Pattern Rules : 1 file .o được tạo từ file.c
%o : %c (bất kỳ)
- 'notdir' - not direct : sẽ xóa đi đường dẫn và giữ lại source file

Bài 11: Kiến thức thêm.

- ifeq vâ export
- + (set) window.
- export MODULE=GPIO
- make print-MODULE
- + ifeq $\Rightarrow$ de' kta module.
- D define
- include Makefile